CASE STUDY · HEALTHCARE SAAS · WORKFLOW EXPERIMENTATION

Optimizing the Coder Workflow, One Experiment at a Time

A structured A/B testing program on the medical coding platform that lifted productivity 5–8% and cut time-per-chart 4–6% — with quality held flat.

+5–8%
PRODUCTIVITY LIFT

−4–6%
TIME PER CHART

1,000+
CODERS IN THE TEST POOL

1M+
CODING ACTIONS POWERING DESIGN

Arun · Senior Product Analytics Associate · athenahealth

Throughput varied wildly — and we didn't know why.

THE BUSINESS MODEL

Coders use our platform to turn clinicians' notes into billing codes. The business depends on two things: throughput (charts per hour) and quality (low rework). Both directly drive customer revenue and ours.

WHAT THE LOGS SHOWED

Event logs revealed wide variation in time-per-chart across coders. Coders complained about queue ordering, error prompts, and note layout — but no one knew which fix would actually move the needle.

Queue ordering felt random

Coders bounced between specialties and case types — context-switching on every chart compounded across a shift.

Error prompts caused friction

Error UI required multiple back-and-forth clicks to resolve — a small drag per chart, painful at scale.

Note layout buried key info

Coders scrolled and hunted for the diagnoses and procedures that drove their decisions — wasted seconds, every chart.

Test the workflow, not just the UI — and never trade quality for speed.

THE PROPOSAL

Don't redesign the coder workflow on opinion. Build a structured A/B testing program across queue ordering, error prompts, and note layout — with quality as a hard guardrail.

PRIMARY METRICS

Charts per hour and time per chart — measured at the coder level from event-log timestamps.

Speed of work, not clicks

QUALITY GUARDRAIL

Error / rework rate. We will not ship a winner that lifts speed and degrades quality.

Healthcare-first discipline

UNIT OF RANDOMIZATION

Coder-level — each coder consistently sees control or treatment to avoid switching confusion.

Workflow-aware design

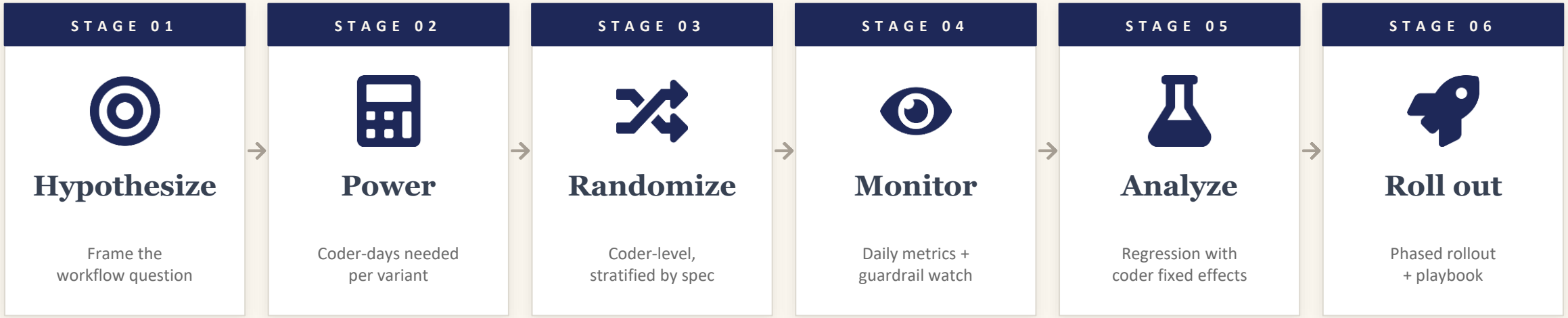
STAKEHOLDERS

Product (decision rights), UX (variant design), Engineering (instrumentation), Operations (rollout).

Cross-functional from day one

Six stages — three parallel experiments, one rigorous framework.

I owned the full experimentation lifecycle and ran three workflow tests in parallel — queue ordering, error prompts, and note layout — under one consistent design.



THE STAFF-LEVEL POINT

I wasn't running one UI test — I was building an experimentation muscle for workflow changes that the team uses today.

Three workflow elements, three hypotheses, three independent tests.

Each experiment isolated one workflow element so we could attribute lift to the change — not to confounding redesigns.

Queue ordering

HYPOTHESIS

Group similar charts → less context switching → faster.

VARIANTS

- Treatment: batch by specialty / case type
- Control: existing FIFO ordering
- Hypothesized lift: 3–5% on charts/hour

Error prompts

HYPOTHESIS

Simpler prompts → fewer back-and-forth clicks.

VARIANTS

- Treatment: redesigned prompt UI + clearer copy
- Control: existing error flow
- Hypothesized lift: reduce clicks-to-resolve

Note layout

HYPOTHESIS

Surface key info higher → less scrolling.

VARIANTS

- Treatment: prominent dx/proc summary at top
- Control: existing note rendering
- Hypothesized lift: 2–4% time/chart

Two primary metrics, one quality guardrail, defined precisely.

METRIC DEFINITIONS (FROM EVENT LOGS)

TIME PER CHART

`t(chart_finalized) − t(chart_open)`

Captures the full handling cycle. Aggregated as median per coder-day to dampen outliers.

CHARTS PER HOUR

`completed_charts ÷ hours_logged`

Throughput at the coder-day level. Hours logged from session events, not self-report.

ERROR / REWORK RATE

`rework_charts ÷ completed_charts`

The quality guardrail — measured on the same coder-day window as productivity.

UPFRONT AGREEMENTS

Aligned with Product, UX, and Operations on these BEFORE we wrote a line of code.

Quality cannot drop

Error/rework rate stays flat or improves — non-negotiable, before metrics, before lift.

Minimum detectable effect

3–4% productivity lift agreed as the bar — anything smaller wasn't worth shipping.

Pre-registered analysis

Final read at the planned end date. No early stopping for positive results.

Stop conditions

Severe negative impact on guardrails trips the kill switch — agreed in advance.

PRINCIPLE · *Faster coding is useless if error rates creep up. Quality guardrails are the first thing we agree on, not the last.*

Coder-level randomization — because workflow tests can't tolerate switching.

WHY NOT CHART-LEVEL?

If a coder bounces between the old and new UI on consecutive charts, every assignment becomes a tax on their attention. The treatment effect gets buried in switching cost.

WHY CODER-LEVEL WORKS

Each coder consistently sees one variant for the whole test. They adapt to it as a real workflow — and the comparison reflects steady-state behavior, not adjustment cost.

DESIGN DECISIONS

Coder-level random hash

Consistent assignment via hash on coder ID — same coder always gets the same variant for the test.

Stratified by specialty

Balanced specialty mix across variants. A win that came from over-sampling easy specialties isn't a win.

Experiment ID + variant on every event

Every chart_open, prompt_shown, and chart_finalized event tagged at the source — no joining needed at analysis time.

PRINCIPLE · *The unit of randomization should match how the user experiences the change. For workflows, that's the user — not the request.*

Powered for 3–4% lift, run for steady-state, not for noise.

POWER CALCULATION INPUTS

INPUT	VALUE	BASIS
Baseline charts/hour	**from 1M+ actions**	*historical mean*
Variance	**log(time/chart)**	*right-skewed*
Minimum detectable effect	**3–4% productivity**	*what's worth shipping*
Significance level (α)	**0.05**	*two-sided*
Power (1 − β)	**0.80 – 0.90**	*comfortable detection*
Unit	**coder-day**	*avoids overweighting*

Required N → required coder-days per variant → translated to a few weeks of run-time given typical coder activity.

RUN-TIME DECISIONS

Cover specialty mix

Run long enough to capture variation across cardiology, primary care, surgical etc. — not just the high-volume specialties.

Cover shift mix

Day shifts and night shifts code differently. Duration set to span enough of each to keep mix balanced.

Pre-registered end date

Locked upfront. No early-stopping for positive results — that's how teams learn to overfit to noise.

Burn-in window excluded

First few days of any variant change can spike adjustment cost. Excluded from the analysis window by design.

PRINCIPLE · *Underpowered tests manufacture noise. The right run-time is what you can defend, not what's convenient.*

Feature flags, instrumentation QA, and a kill-switch dashboard.

IMPLEMENTATION

1 Variants behind feature flags

Worked with UX and Engineering to wire up queue ordering, error prompt, and note layout variants — each independently togglable.

2 Event logging QA

Verified chart_open, prompt_shown, click events, chart_finalized, and rework events all carry coder ID, experiment ID, and variant.

3 Pre-launch QA in staging

Confirmed ~50/50 split, no skew across specialties, and that no event was being silently dropped on the treatment arm.

DAILY MONITORING

Charts/hour & time-per-chart by variant

Daily tracking — watching for divergence and for unexpected mid-test reversals that could signal a bug.

Error/rework rate — guardrail

Tracked equally throughout. Quality is the kill-switch metric — not a footnote.

Coder logout rate

Secondary signal of frustration. A spike could mean the new UI is alienating coders even if speed looks fine.

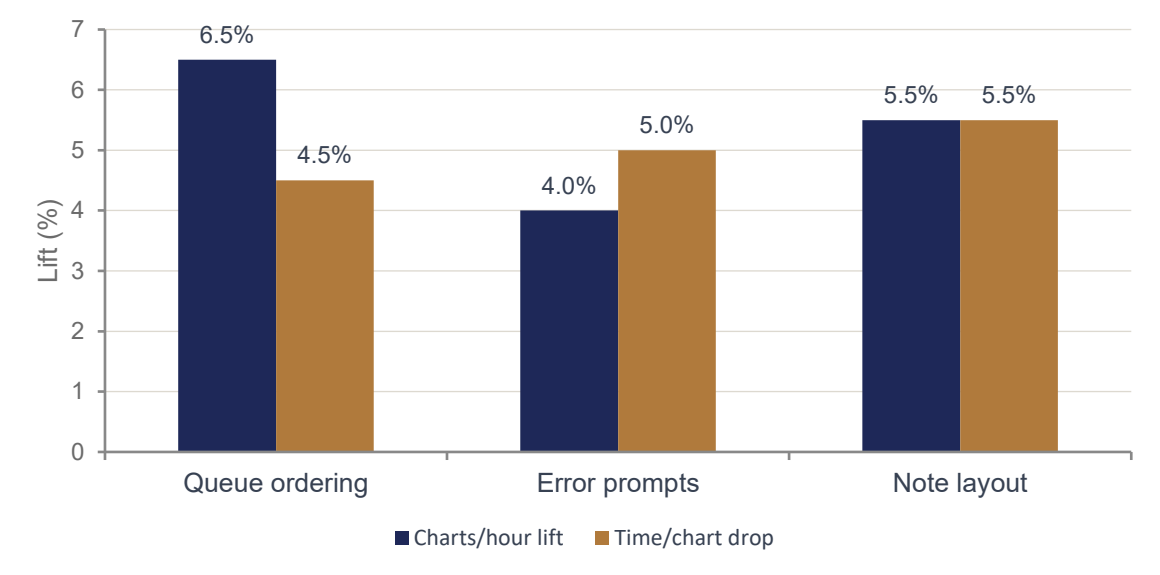
Sample-balance check

Daily check that 50/50 split holds across specialties and shifts — drift here invalidates the analysis.

GUARDRAIL · *Monitoring catches bugs and trip-wires — not significance. Final reads happen at pre-registered end dates.*

Regression with coder fixed effects — not just a t-test.

ESTIMATED LIFT BY EXPERIMENT



Illustrative — point estimates from regression with coder fixed effects + specialty controls.

ANALYSIS DISCIPLINE

Regression with coder fixed effects

A simple t-test ignores that fast coders cluster within variants. Coder fixed effects net out per-coder skill.

Coder-day as the unit

Aggregating to coder-day prevents heavy users from dominating the result. One coder, one observation per day.

Heterogeneity by specialty & shift

Sliced effects by specialty and shift to confirm the lift wasn't isolated to one easy segment.

Quality guardrail held flat

Error/rework rate confirmed flat or improved. No win shipped if quality budged.

OUTCOME · Best variants together: 5–8% productivity lift, 4–6% time-per-chart drop, error rate held flat.

Phased rollout — and a playbook for the next workflow change.

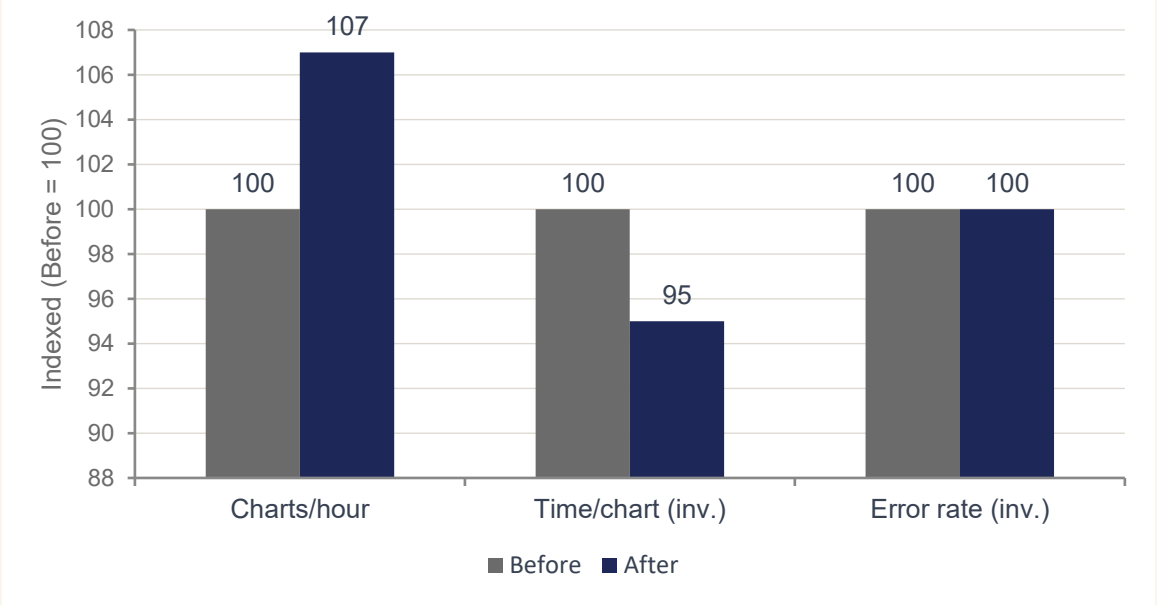
THE ROLLOUT

Phased — winning variants first promoted to the pilot organizations that had participated in tests, then expanded to the full coder base after gains held in production.

WHAT MOVED

- Productivity (charts/hour) — primary outcome
- Time per chart — secondary outcome
- Error rate — held flat (the win condition)
- Coder qualitative feedback — "got out of the way"

BEFORE vs. AFTER ROLLOUT



Time/chart and error rate shown as inverse so direction is consistent (higher = better).

HEADLINE IMPACT

+5 to +8% productivity · −4 to −6% time per chart — with error rate held flat across the coder base.

The scoreboard.

PRODUCTIVITY LIFT

+5 to +8%

charts per hour, primary outcome held in production

TIME PER CHART

−4 to −6%

secondary outcome — workflow felt smoother to coders

CODERS COVERED

1,000+

winning variants now defaults across the coder base

DURABLE DELIVERABLE

Playbook

next workflow change defaults to A/B framework

QUALITATIVE WINS

Coders reported the system "got out of the way" · Quality guardrails held — no shipped speed at the expense of accuracy · Experimentation framework now standard for workflow changes.

What worked, what I'd extend next.

WHAT WORKED

Three independent tests, one framework

Running queue, prompts, and layout in parallel under one design pattern multiplied learning per unit of effort.

Coder-level randomization

Switching costs would have buried the treatment effect in chart-level designs. The randomization unit choice was the test design choice.

Quality as a hard guardrail

Pre-agreeing that no speed win would ship if errors moved built trust — and made the wins defensible to leadership.

WHAT I'D EXTEND NEXT

Personalized queues by coder

Now that batch-by-specialty wins, the next step is personalized queues that learn each coder's strengths and rotate workload accordingly.

Multi-armed bandit for prompts

For lower-stakes UI tweaks like prompt copy, bandits can balance learning and traffic optimization — faster than another full A/B.

Cross-test interaction effects

Three winners tested independently. Worth a follow-up to verify the combined effect — sometimes wins cancel rather than compound.

Workflow optimization as an experimentation muscle.

01 **Match the unit of randomization to the user's reality.**

Workflow tests need user-level randomization. The wrong unit choice will bury the effect you're trying to measure — every time.

02 **Quality is the first metric, not the last.**

In healthcare especially, faster work that creates more rework is a regression dressed up as a win. Guardrails go in the design, not the analysis.

03 **The deliverable is the muscle.**

One workflow win is nice. A team that defaults to A/B-testing the next workflow change is what compounds — and what makes this Staff-level work.

Thank you · Questions?